INTERNSHIP PROJECT REPORT

TASK 2

INTELLIGENT MULTI-CLASS NATURAL LANGUAGE TEXT SENTIMENT CLASSIFIER

Progree AI Internship Program

By

Name: Shazmeen Irshad

Student ID: B9/2929

1. Abstract:

Sentiment analysis is an important Natural Language Processing (NLP) task that enables computers to identify opinions and emotions expressed in textual data. This project focuses on developing an intelligent multi-class sentiment classifier capable of categorizing text into positive, negative, and neutral sentiments. The system was developed using Python and Scikit-Learn in Google Colab. The Twitter US Airline Sentiment dataset was used for training and evaluation. Text preprocessing techniques, including lowercase conversion, special character removal, stopword removal, and lemmatization, were applied to prepare the data. TF-IDF was used to transform text into numerical features, while Logistic Regression was selected as the classification algorithm. The trained model achieved an accuracy of **76.20%** and a macro F1-score of **71.53%** on the testing dataset. The project demonstrates a complete machine learning workflow, including data preprocessing, feature extraction, model training, evaluation, prediction, and model saving.

2. Introduction:

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, process, and analyze human language. Sentiment analysis is one of the applications of NLP. It identifies the emotional tone or opinion expressed in text. Sentiment analysis is commonly used in customer feedback analysis, product reviews, social media monitoring, and business decision-making.

The purpose of this project is to build a machine learning-based sentiment classification system that automatically classifies text into three categories:

- Positive
- Negative
- Neutral

The system uses text preprocessing techniques, TF-IDF feature extraction, and Logistic Regression to generate sentiment predictions.

3. Problem Statement:

A large amount of textual data is generated daily through social media, customer reviews, and online discussions. Manually analyzing this information is time-consuming and difficult. The problem addressed in this project is to develop an automated system that can analyze textual

statements and identify their sentiment. The system should be capable of processing text, extracting useful features, and predicting whether the sentiment is positive, negative, or neutral.

4. Project Objectives:

The main objectives of this project are:

- To develop a multi-class sentiment classification model.
- To classify text into positive, negative, and neutral categories.
- To preprocess text using NLP techniques.
- To remove stopwords and apply lemmatization.
- To convert text into numerical features using TF-IDF.
- To train a machine learning model using Scikit-Learn.
- To evaluate model performance using accuracy and F1-score.
- To generate sentiment predictions for custom text.
- To save the trained model for future use.

5. Scope of the Project:

The project focuses on sentiment classification of English-language text, particularly airline-related tweets.

The system includes the following functionalities:

- Dataset loading and preparation.
- Text cleaning and preprocessing.
- Feature extraction using TF-IDF.
- Machine learning model training.
- Sentiment classification.
- Model performance evaluation.
- Custom text prediction.
- Saving the trained model and vectorizer.

The current implementation is designed as a machine learning prototype and can be extended into a web-based sentiment analysis application in the future.

6. Tools and Technologies:

The following technologies were used to develop the project.

Technology	Purpose
Python	Main programming language
Google Colab	Development and execution environment
Pandas	Dataset loading and manipulation
NumPy	Numerical operations
NLTK	Text preprocessing and lemmatization
Scikit-Learn	Machine learning and evaluation
TF-IDF Vectorizer	Text feature extraction
Logistic Regression	Sentiment classification
Matplotlib	Data visualization
Seaborn	Confusion matrix visualization
Joblib	Saving the trained model

7. Dataset Description:

The project uses the Twitter US Airline Sentiment dataset, which contains tweets related to airline services and customer experiences. The dataset includes text messages and corresponding sentiment labels.

The sentiment classes are:

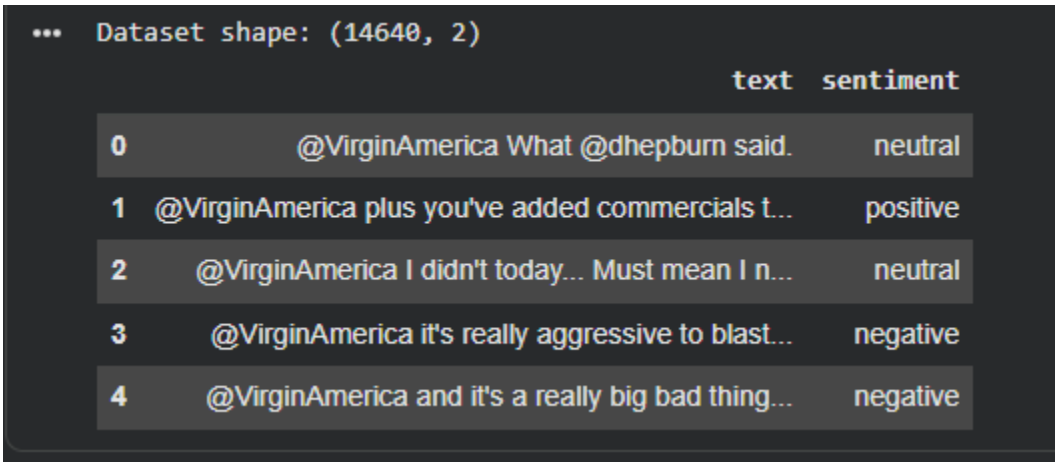
- Positive
- Negative
- Neutral

The dataset was loaded from a publicly available CSV file using Pandas. After selecting the relevant columns and removing missing values, the dataset contained 14,640 text samples.

Dataset distribution:

Sentiment	Number of Samples
Negative	9,178
Neutral	3,099
Positive	2,363
Total	**14,640**

The dataset contains more negative tweets than positive and neutral tweets. Therefore, class imbalance was considered during model training.



```
Dataset shape: (14640, 2)
```

	text	sentiment
0	@VirginAmerica What @dhepburn said.	neutral
1	@VirginAmerica plus you've added commercials t...	positive
2	@VirginAmerica I didn't today... Must mean I n...	neutral
3	@VirginAmerica it's really aggressive to blast...	negative
4	@VirginAmerica and it's a really big bad thing...	negative

8. Methodology:

The project follows a structured machine learning workflow.

The overall process consists of:

1. Dataset loading
2. Data preparation
3. Text preprocessing
4. Exploratory data analysis
5. Train-test splitting
6. TF-IDF feature extraction
7. Logistic Regression training
8. Model evaluation
9. Custom sentiment prediction
10. Model saving

The processed text is converted into numerical representations before being passed to the machine learning classifier.

Project workflow:

Dataset → Preprocessing → TF-IDF → Logistic Regression → Evaluation → Prediction

9. Data Preprocessing:

Text preprocessing is an important step in NLP because raw text often contains unnecessary characters, inconsistent capitalization, and words that may not contribute significantly to classification.

The following preprocessing operations were applied.

1. **Lowercase Conversion:**

All text was converted into lowercase to ensure that words with different capitalization were treated consistently.

For example:

```
Amazing Service
```

becomes:

```
amazing service
```

2. **Special Character Removal:**

Special characters and numbers were removed using regular expressions. This step helped reduce unnecessary symbols in the text.

3. **Tokenization:**

The text was split into individual words using whitespace-based splitting.

4. **Stopword Removal:**

Common English stopwords were removed using the NLTK stopword list. Examples include common words such as “the”, “is”, and “a”.

5. **Lemmatization:**

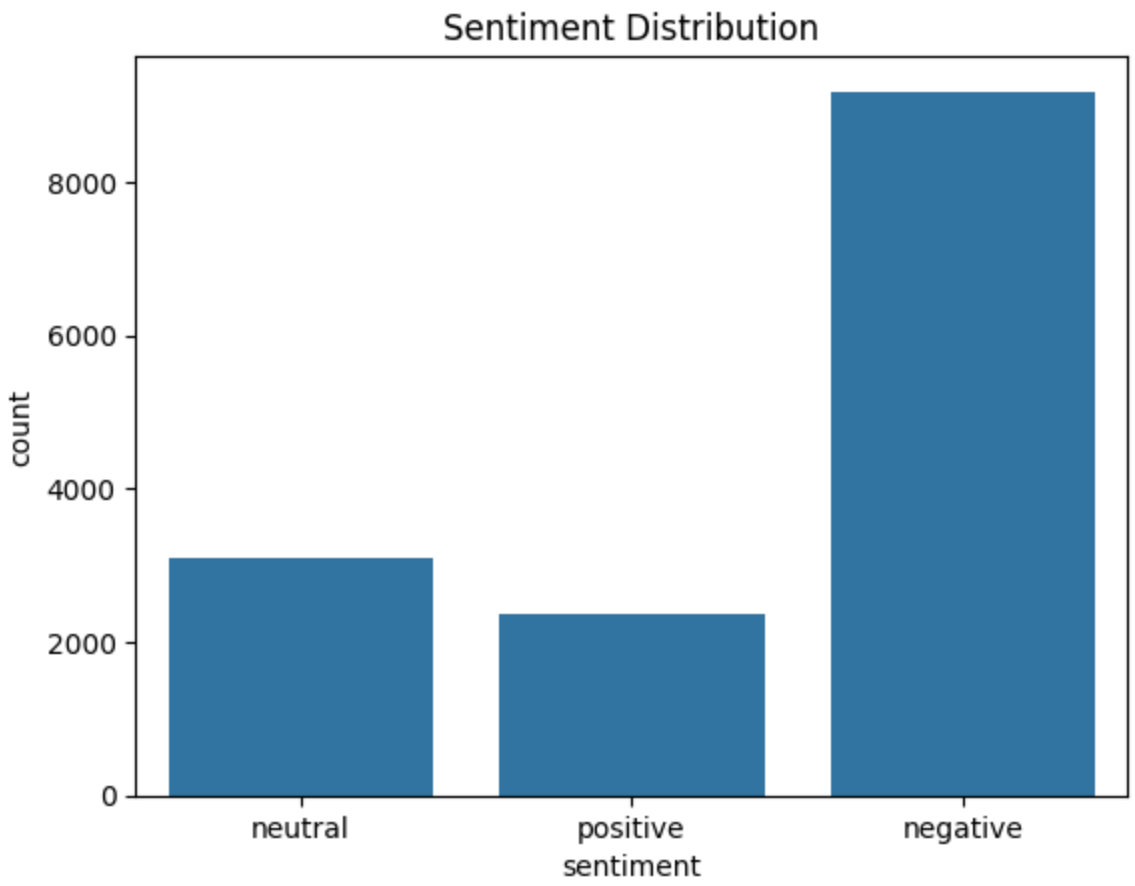
WordNet Lemmatizer was used to convert words into their base forms. For example, different word forms may be reduced to a common base form. This process helps reduce vocabulary complexity.

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

	text	clean_text	sentiment
0	@VirginAmerica What @dhepburn said.	virginamerica dhepburn said	neutral
1	@VirginAmerica plus you've added commercials t...	virginamerica plus added commercial experience...	positive
2	@VirginAmerica I didn't today... Must mean I n...	virginamerica today must mean need take anothe...	neutral
3	@VirginAmerica it's really aggressive to blast...	virginamerica really aggressive blast obnoxiou...	negative
4	@VirginAmerica and it's a really big bad thing...	virginamerica really big bad thing	negative

10. Exploratory Data Analysis:

Exploratory Data Analysis (EDA) was performed to understand the distribution of sentiment labels. A bar chart was generated to visualize the number of tweets in each sentiment category. The visualization showed that negative sentiment was the dominant class in the dataset. This observation indicated that the dataset was imbalanced. To address this issue during model training, balanced class weights were used in Logistic Regression.



11. Train-Test Split:

The dataset was divided into training and testing subsets. The training dataset was used to train the model, while the testing dataset was used to evaluate its performance on unseen samples.

The dataset was split using the Scikit-Learn **`train_test_split()`** function.

The split configuration was:

- Training data: 80%
- Testing data: 20%
- Random state: 42
- Stratification: Enabled

The resulting dataset sizes were:

Dataset	Samples
Training dataset	11,712
Testing dataset	2,928
Total	14,640

Stratification was used to preserve the sentiment class distribution in both subsets.

```
Training samples: 11712
Testing samples: 2928
```

12. Feature Extraction Using TF-IDF:

Machine learning algorithms require numerical input rather than raw text. TF-IDF (Term Frequency–Inverse Document Frequency) was used to convert the cleaned tweets into numerical feature vectors. TF-IDF assigns importance to words based on their frequency in a document and how common they are across the dataset.

The vectorizer was configured with the following parameters:

Parameter	Value
Maximum features	10,000
N-gram range	(1, 2)

The n-gram range allows the model to consider:

- Individual words (unigrams)
- Two-word combinations (bigrams)

The TF-IDF vectorizer was fitted on the training dataset and then applied to the testing dataset. This ensured that feature extraction was learned from the training data.

13. Model Selection and Training:

Logistic Regression was selected as the machine learning algorithm for sentiment classification. Logistic Regression is a supervised learning algorithm commonly used for classification tasks. It estimates the probability that an input belongs to a particular class. In this project, it was used to classify text into three sentiment categories.

The model was configured with:

Parameter	Value
Algorithm	Logistic Regression
Maximum iterations	1,000
Class weights	Balanced

The `class_weight='balanced'` setting was used to help compensate for differences in the number of samples across sentiment classes.

The model was trained using the TF-IDF features extracted from the training dataset.

```
# Initialize Logistic Regression model

model = LogisticRegression(
    max_iter=1000,
    class_weight='balanced'
)

# Train the model

model.fit(X_train_tfidf, y_train)

print("Sentiment classification model trained successfully!")
```

```
Sentiment classification model trained successfully!
```

14. Model Evaluation:

After training, the model was evaluated using the testing dataset.

The following metrics were calculated:

1. Accuracy:

Accuracy measures the proportion of correctly classified samples out of all testing samples.

The model achieved an accuracy of:

76.20%

2. F1-score:

The F1-score combines precision and recall into a single metric. The macro F1-score calculates the average F1-score across all classes, giving each sentiment class equal importance.

The model achieved a macro F1-score of:

71.53%

Evaluation Results

Metric	Score
Accuracy	76.20%
Macro F1-score	71.53%

The results indicate that the model was able to classify sentiment reasonably well on the testing dataset.

```
Model Accuracy: 0.762
Macro F1-Score: 0.7153
```

15. Classification Report:

A classification report was generated to evaluate the performance of the model for each sentiment class.

The report includes precision, recall, F1-score, and support.

The obtained results were:

Classification Report:

	precision	recall	f1-score	support
negative	0.90	0.80	0.84	1835
neutral	0.54	0.70	0.61	620
positive	0.68	0.71	0.69	473
accuracy			0.76	2928
macro avg	0.70	0.74	0.72	2928
weighted avg	0.79	0.76	0.77	2928

Interpretation:

The model performed best on negative tweets, achieving an F1-score of 0.84. Neutral sentiment was comparatively more difficult to classify, with an F1-score of 0.61. The model also demonstrated reasonable performance on positive tweets, achieving an F1-score of 0.69.

These results indicate that the classifier performs differently across sentiment categories, partly because the dataset contains an unequal number of samples in each class.

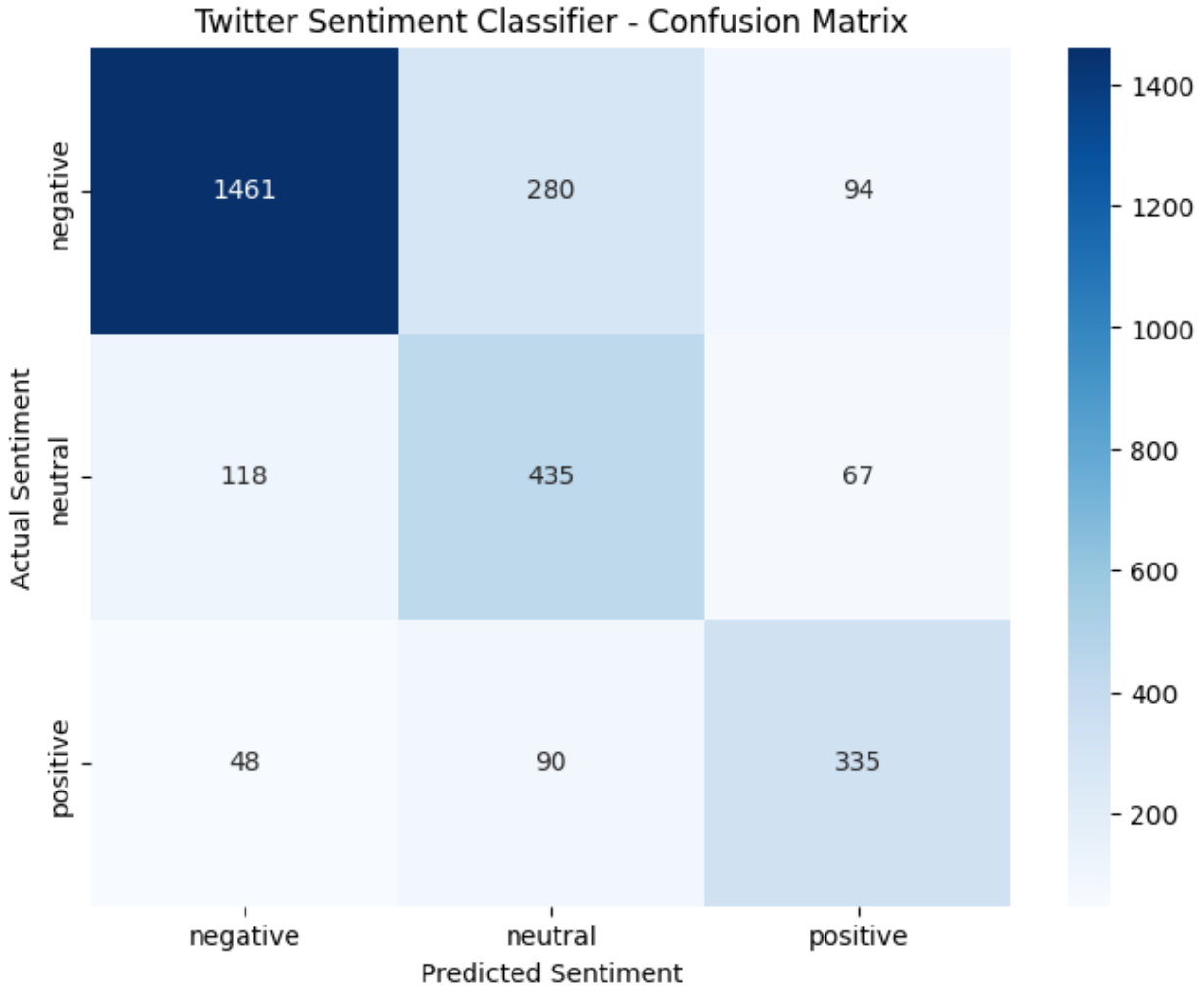
16. Confusion Matrix:

A confusion matrix was generated to visualize the classification results. The confusion matrix compares the actual sentiment labels with the predicted sentiment labels.

It helps identify:

- Correctly classified negative tweets.
- Correctly classified neutral tweets.
- Correctly classified positive tweets.
- Sentiment classes that are frequently confused with one another.

The confusion matrix provides a more detailed view of classification performance than accuracy alone.



17. Sentiment Prediction on Custom Text:

The trained classifier was tested on custom text inputs to demonstrate its prediction functionality.

A prediction function was created to:

1. Accept a text input.
2. Apply the preprocessing function.
3. Convert the cleaned text into TF-IDF features.
4. Pass the features to the trained Logistic Regression model.
5. Return the predicted sentiment.

Example input sentences used in testing:

Example 1:

“I absolutely love this airline, the service is amazing!”

Example 2:

“My flight was delayed and the staff were very rude.”

Example 3:

“My flight is scheduled for tomorrow morning.”

The model generated sentiment predictions for these text inputs.

```
Tweet: I absolutely love this airline, the service is amazing!
Predicted Sentiment: positive
--------------------------------------------------
Tweet: My flight was delayed and the staff were very rude.
Predicted Sentiment: negative
--------------------------------------------------
Tweet: My flight is scheduled for tomorrow morning.
Predicted Sentiment: neutral
--------------------------------------------------
```

18. Model Saving and Reusability:

After training and evaluation, the trained model and TF-IDF vectorizer were saved using Joblib.

The following files were generated:

- `sentiment_model.pkl`
- `tfidf_vectorizer.pkl`

The model file stores the trained Logistic Regression classifier.

The vectorizer file stores the TF-IDF transformation settings learned from the training data.

Both files are required to make predictions on new text without retraining the model.

Saving the model makes the project reusable and allows future integration into applications such as Flask or FastAPI.

```
Model and TF-IDF vectorizer saved successfully!
```

19. Limitations:

Although the model achieved reasonable classification performance, some limitations remain.

1. Dataset Domain:

The model was trained on airline-related tweets. It may not perform equally well on text related to other domains such as healthcare, education, or product reviews.

2. Class Imbalance:

The dataset contains more negative tweets than neutral and positive tweets. Although balanced class weights were used, class imbalance may still affect classification performance.

3. Text Preprocessing:

Removing stopwords can sometimes remove words that are important for sentiment, particularly negation words such as “not” and “never”. This may affect the interpretation of certain sentences.

4. Model Complexity:

Logistic Regression is a traditional machine learning algorithm. More advanced NLP models may improve classification performance but require additional computational resources and experimentation.

20. Future Improvements:

The project can be improved in several ways:

- Preserve important negation words during preprocessing.
- Experiment with other machine learning algorithms such as Support Vector Machines.
- Tune hyperparameters using cross-validation.
- Evaluate the model on datasets from different domains.

- Use advanced NLP models such as BERT.
- Develop a web-based interface using Flask or FastAPI.
- Allow users to enter text and view sentiment predictions through a browser.
- Add confidence scores for predictions.

These improvements may increase classification performance and make the system more practical for real-world applications.

21. Conclusion:

The Intelligent Multi-Class Natural Language Text Sentiment Classifier was successfully developed using Python, NLTK, TF-IDF, and Logistic Regression. The project implemented a complete machine learning pipeline, including dataset loading, text preprocessing, feature extraction, model training, evaluation, custom prediction, and model saving. The classifier achieved an accuracy of **76.20%** and a macro F1-score of **71.53%** on the testing dataset. The model demonstrated its ability to classify airline-related tweets into positive, negative, and neutral sentiment categories. This project provided practical experience in Natural Language Processing, text preprocessing, feature engineering, machine learning classification, and model evaluation.